Snowflake End-to-End: Real-Time Problems and Agent Playbook

snowflake-ai-data-agent · 158 problems data engineers and analysts hit on this stack, the agent that handles each one, and how to train those agents

What this document is

The stack is Alpaca (trades, bars, news) → Kafka → Snowpipe Streaming and GCS batch loads → Bronze → Streams and Tasks → Silver → dbt → Gold → Semantic View and Cortex Search → Cortex Agent → MCP, Streamlit and FastAPI. Every stage has its own way of failing quietly. This playbook lists those failures as engineers and analysts actually meet them, assigns each one to an agent, and sets out how each agent is trained and tested before it is trusted.

> **Approach:** the agent roles below are *responsibilities*, not a fleet of custom agents to build. Existing tools fill most of them (Snowflake alerts and Tasks, Dagster checks, dbt tests, Terraform, CI, and the Snowflake Cortex Agent); this project writes the glue between those tools and the guardrails around them, with the goal of removing repetitive human work while keeping human approval for anything that changes money, access or production (docs/production-readiness.md). **Status:** only the DLQ routing, sample data and ONNX training are verified; everything else is written but has not run against live Snowflake.

Who feels which problems

- **Data engineers** own freshness, completeness, capacity and cost: feeds that stall, servers that go down, volume spikes that overload every component, timestamps that don't line up, tasks that suspend, tests that fail overnight, grants that disappear, deploys that break production, role sprawl and surprise bills.
- **Analysts** own trust in the numbers: ambiguous definitions, confident wrong SQL, slow queries, not finding the right table, delayed data that looks broken, news citations that point to the wrong company, and waiting on engineering for every new metric.
- Many problems belong to both: a data engineer's silent NULL becomes an analyst's wrong average.

What “training” means for these agents

Most of these agents are LLM agents. They get better through what they are given and how they are tested, not by retraining model weights. Training each agent has five parts:

- **Knowledge.** Runbooks, schemas, metric definitions and error catalogues, written down in the repo.
- **Tools.** A small set of narrow, mostly read-only tools (listed per agent below). Anything that changes state is approval-gated.
- **Scenarios.** Faults injected on purpose, using the repo's own sample-data generators and replay script, so every problem in this document becomes a repeatable test case with a known right answer.
- **Evals.** A scored test set per agent, run on every change to its instructions or tools. An agent is promoted only when it passes.
- **Feedback.** Every human correction in production becomes a new eval case, so the same mistake is caught next time.

Real model training is used in only two places: the anomaly-detection model (the IsolationForest exported to ONNX, already in snowpark/) and, if evals plateau, fine-tuning for text-to-SQL or routing. Fine-tuning comes last, not first.

How the agents understand these problems

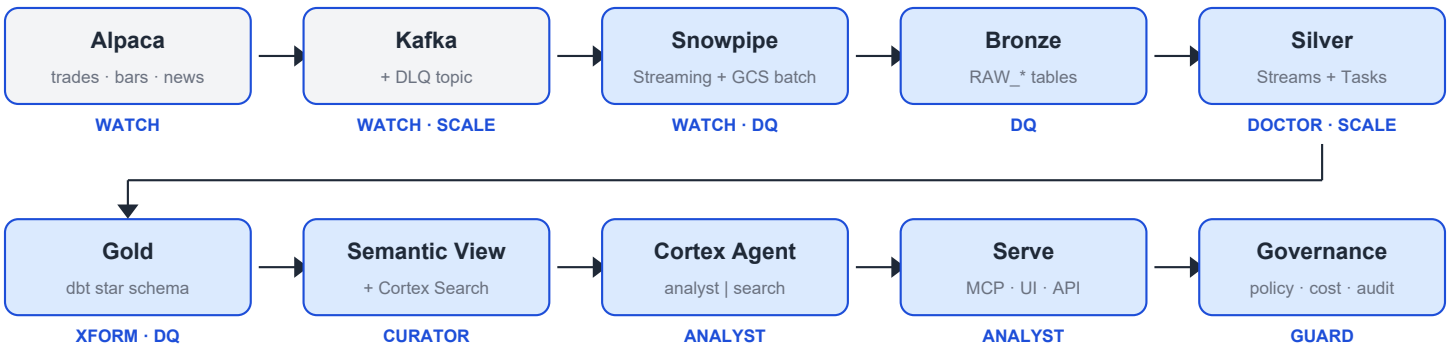
Every problem in this document comes from one machine-readable file, knowledge/problem_catalog.yaml, which also holds the agent roster and training plans. The agents load it as knowledge: when they see a symptom they match it to a problem ID, follow that entry's solution, and stay inside its guardrail. The PDF is rendered from the same file, so what people read and what the agents know can't drift apart. Adding a problem means adding one entry; the build checks that every entry names a real agent and that every agent has a training plan.

Autonomy levels

Level	What the agent may do	Used for
L0 Observe	Read metrics and logs, raise alerts.	Every agent on day one.
L1 Diagnose	Explain root cause with evidence; recommend a fix.	Default level for most problems.
L2 Act with approval	Run a runbook step after a human clicks approve.	Restarts, backfills, resuming tasks.
L3 Act within bounds	Run a pre-approved runbook step automatically, once, then escalate.	Producer restart, alert suppression on closed markets.

Snowflake end to end

The whole pipeline, from the market feed to the analyst's answer. Everything after Kafka runs inside Snowflake, so most problems can be detected, diagnosed and fixed with Snowflake's own objects and views. Problem IDs refer to the tables later in this document.



Grey = outside Snowflake · Blue = inside Snowflake · Labels under boxes = agent watching that stage. TRIAGE and SCALE cover all of them.

Stage	Snowflake objects in this repo	Problems	Agent	Signal the agent reads
1. Source	Outside Snowflake: Alpaca websocket + REST; Kafka topics market-trades, market-bars, market-data-dlq	A1–A9, B1–B5, I1–I4, J1–J3, K2, L1	WATCH, SCALE	Kafka lag + Connect status and Alpaca clock (external); results written into a Snowflake OPS table
2. Ingest	Snowpipe Streaming channels (Kafka connector); GCS storage integration, RAW_STAGE, BARS_BACKFILL_PIPE, SYMBOLS_PIPE, NEWS_PIPE	C1–C5, I5, J4, J6, J7, K7	WATCH, DQ, SCALE	SNOWPIPE_STREAMING_* history, COPY_HISTORY, SYSTEM$PIPE_STATUS
3. Bronze	BRONZE.RAW_TRADES, RAW_BARS, RAW_SYMBOLS, RAW_NEWS (permissive VARCHAR)	A3, A8, C1, D4, J8, K1, L2	DQ	Data Metric Functions: ROW_COUNT, DUPLICATE_COUNT, FRESHNESS
4. Silver	Streams RAW_*_STREAM + Tasks MERGE_*_TASK → SILVER.TRADES, BARS, SYMBOLS, NEWS; SYMBOLS_DT	D1–D5, I6, K5, K6, M1, M2	DOCTOR, SCALE	SHOW STREAMS (stale_after), TASK_HISTORY, DYNAMIC_TABLE_REFRESH_HISTORY
5. Gold	dbt → STAGING views; GOLD.DIM_SYMBOLS, DIM_DATE, FCT_TRADES, FCT_BARS (is_suspect flag; row access policy via post-hook)	E1–E6, A5, A6, A9, K3, K4, K10, L3–L6, M3–M6, J9	DOCTOR, XFORM, DQ	dbt run_results, POLICY_REFERENCES, NULL_COUNT DMFs, ONNX anomaly score
6. Semantic + search	GOLD.MARKET_SEMANTIC_VIEW; semantic_model.yaml; SILVER.NEWS_SEARCH_SERVICE (Cortex Search)	F2–F5, F8, N2, N4	CURATOR	Verified-query regression against sql/08; search refresh lag
7. Agent	AGENTS.MARKET_DATA_AGENT (Cortex Agent with analyst + search tools), role ANALYST_AGENT	F1–F9, I8, J5, K9, N1, N3, N6	ANALYST	GOVERNANCE.AGENT_QUERY_AUDIT_LOG, Cortex usage views

Stage	Snowflake objects in this repo	Problems	Agent	Signal the agent reads
8. Serve	MCP server, Streamlit, FastAPI, all querying as ANALYST_AGENT through the same policy	F6, F9, G2, I7, N5	ANALYST, GUARD, SCALE	Latency and error logs in an event table
9. Govern + cost	Roles, DELAYED_DATA_POLICY, ANALYST_WH, ANALYST_AGENT_MONITOR, audit log	G1–G5, E4, I9, K8, S1–S9, T1–T8	GUARD	GRANTS_TO_ROLES, ACCESS_HISTORY, WAREHOUSE_METERING_HISTORY, resource monitors
10. SQL everywhere	Every Silver MERGE, dbt model, verified query, analyst worksheet and agent-generated query	P1–P14, Q1–Q11	SQLCOACH	EXPLAIN, GET_QUERY_OPERATOR_STATS, QUERY_HISTORY (spill, partitions scanned, rows produced)
11. Develop → deploy	Git, dbt CI, DEV/TEST/PROD databases (zero-copy clones), migration scripts	R1–R12	RELEASE	CI results, GET_DDL diffs, OBJECT_DEPENDENCIES, ACCESS_HISTORY
12. Analyst self-service	Snowsight worksheets, BI tools, certified Gold objects, column comments	U1–U9	ANALYST, CURATOR	ACCESS_HISTORY, QUERY_HISTORY by user, permission errors

The agent roster

11 roles, each with one job. Each is filled first by an existing tool (named in its tools column) plus our checks; an AI assistant is added only where a human would otherwise do repetitive diagnosis or drafting. ANALYST is the Cortex Agent the repo already defines in sql/10_agent.

Agent	Owns	Tools it is given
WATCH Ingestion Watchdog	Alpaca feed, Kafka, Kafka Connect, Snowpipe Streaming, GCS/Snowpipe batch loads	Alpaca /v2/clock + /v2/calendar; Kafka consumer-group lag and topic offsets (read-only admin API); Kafka Connect REST status; SNOWPIPE_STREAMING_CLIENT/CHANNEL history; SYSTEM$PIPE_STATUS; COPY_HISTORY; DLQ topic reader
DOCTOR Pipeline Doctor	Streams, Tasks, Dynamic Tables, dbt runs	SHOW STREAMS (stale flag); TASK_HISTORY; DYNAMIC_TABLE_REFRESH_HISTORY; dbt run_results.json + manifest.json; EXECUTE TASK / RESUME TASK (approval-gated)
DQ Data Quality Sentinel	Correctness of the numbers in Bronze, Silver and Gold	Row-count reconciliation queries across layers; FCT_BARS.is_suspect; ONNX anomaly model score; Alpaca corporate-actions API; sql/08_validation queries
XFORM Transformation Copilot	dbt models, schema changes, tests, docs	Git repo (read + open PR only); dbt parse/compile/test against a dev schema; INFORMATION_SCHEMA.COLUMNS diff
ANALYST Analyst Copilot	Business questions from humans (this is the Cortex Agent the repo already defines)	Cortex Analyst over MARKET_SEMANTIC_VIEW; Cortex Search over NEWS_SEARCH_SERVICE; freshness lookup on Gold
CURATOR Semantic Curator	Semantic model quality: metrics, synonyms, verified queries	semantic_model.yaml + semantic_view.sql (read + PR); AGENT_QUERY_AUDIT_LOG; thumbs-down feedback table
GUARD Governance and Cost Guard	Access, licensing, policy drift, credit spend, prompt injection	POLICY_REFERENCES; GRANTS_TO_ROLES; ACCESS_HISTORY; WAREHOUSE_METERING_HISTORY; CORTEX usage views; resource monitors
SCALE Capacity and Resilience	Load spikes, scaling, outages, failover and catch-up after recovery	WAREHOUSE_LOAD_HISTORY; QUERY_HISTORY queued/blocked time; Kafka broker disk and throughput metrics; connector task counts; replication/failover group status; ALTER WAREHOUSE within fixed size limits (pre-approved)
SQLCOACH SQL Coach	Correctness and performance of every SQL statement: Silver MERGEs, dbt models, verified queries, analysts' worksheets and SQL the ANALYST agent generates	EXPLAIN; GET_QUERY_OPERATOR_STATS (query profile); QUERY_HISTORY (spill, partitions scanned, rows produced); run a query and its rewrite on a zero-copy clone and compare results; this catalog's bad/good SQL pairs
RELEASE Release Manager	Getting changes from a developer's branch into production safely: environments, migrations, CI, deploy order, rollback, impact on consumers	Git + CI results; dbt manifest and state comparison; GET_DDL diffs between DEV/TEST/PROD; OBJECT_DEPENDENCIES; ACCESS_HISTORY (who reads a column); CREATE DATABASE ... CLONE; ALTER TABLE ... SWAP WITH (approval-gated)
TRIAGE Incident Triage (orchestrator)	Any alert; decides which specialist agent handles it	Calls the other agents as tools; writes the incident timeline; pages a human with a summary

The problems (158), stage by stage

Each row gives the problem as it shows up, who feels it, how the assigned agent handles it, and the guardrail or human step that keeps the agent safe.

A. Market-data source (Alpaca websocket + REST)

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
A1	**Silent stall.** Websocket stays connected but stops sending messages, so the pipeline looks healthy while prices freeze.	DE	**WATCH** compares last-message age against the Alpaca market clock: no messages for 60 s while the market is open is an incident. Restarts the producer through its runbook.	Auto-restart allowed once; a second stall pages a human.
A2	**“No data” vs “broken”.** Weekends, holidays, half-days and pre-market look identical to an outage. Alerts fire every Saturday.	DE	**WATCH** checks /v2/calendar before raising anything. Suppresses alerts outside trading sessions and says so in the alert text.	Calendar cached daily; a mismatch with exchange calendar is flagged.
A3	**Reconnect duplicates and gaps.** After a reconnect some trades repeat and some minutes are missing.	DE	**DQ** finds missing minute bars per symbol and duplicate trade_ids, then asks **WATCH** to backfill the exact window via REST.	Backfill limited to the gap window; never rewrites outside it.
A4	**Rate limits (HTTP 429) during backfill.** Large history pulls fail halfway.	DE	**WATCH** reads the failure, re-plans the backfill into smaller symbol/date chunks, resumes from the last successful chunk.	Concurrency cap set in config, not chosen by the agent.
A5	**Corporate actions.** A 4:1 split makes a stock look like it fell 75%. Ticker renames (FB to META) split history in two.	DE + Analyst	**DQ** joins suspect moves against the corporate-actions API. Labels split-driven moves as explained, proposes an adjustment-factor table.	Price adjustment is a reviewed PR, never an in-place update.
A6	**Trade corrections and cancels.** Exchanges cancel or correct prints; condition codes mark odd-lot or out-of-sequence trades.	DE + Analyst	**XFORM** proposes a condition-code filter in stg_trades; **DQ** reports how many trades each rule removes.	Filter rules documented and tested in dbt.
A7	**Feed coverage mismatch.** Free Alpaca IEX feed is a small share of total volume. Analysts compare to Bloomberg and think the data is wrong.	Analyst	**ANALYST** states the feed source in every volume answer and explains the gap when a user disputes a number.	Feed name stored as a column, not assumed.
A8	**Vendor schema drift.** A new field appears or a type changes, and validation starts rejecting every message.	DE	**WATCH** sees the DLQ rate spike, clusters DLQ errors by field, and hands a summary to **XFORM** to draft a schema update.	Schema change goes through PR + CI; DLQ is replayed after merge.
A9	**Timezone and DST bugs.** ET vs UTC confusion shifts the trading day; the DST weekend breaks “today”.	DE + Analyst	**DQ** asserts that every session has 390 minute bars between 13:30/14:30 UTC and 20:00/21:00 UTC depending on DST.	Assertion lives in dbt tests, not only in the agent.

B. Kafka and Kafka Connect

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
B1	**Consumer lag growing.** Connector falls behind; “real-time” data becomes minutes old.	DE	**WATCH** tracks lag trend, not a single value. Diagnoses cause (connector down, hot partition, slow Snowflake) and recommends a fix.	Scaling tasks or partitions needs approval.
B2	**DLQ grows silently.** Bad messages are routed correctly, then nobody ever looks at them.	DE	**WATCH** summarises the DLQ daily: top error types, affected symbols, whether the rows are replayable.	Replay of DLQ only after the root cause is fixed.
B3	**Hot partitions.** TSLA/NVDA traffic lands on one partition and lags while others idle.	DE	**WATCH** reports per-partition lag and proposes a different keying strategy.	Repartitioning is a planned change, never automatic.
B4	**Retention expires before catch-up.** A long outage means Kafka deletes data that never reached Snowflake.	DE	**WATCH** projects time-to-retention-expiry from current lag and escalates early; **TRIAGE** triggers REST backfill for the lost window.	Escalation threshold set as a fixed fraction of retention.
B5	**Connector task failures.** A task dies with a stack trace; restarts loop.	DE	**WATCH** reads the Connect REST trace, matches it to known causes (auth, channel invalidated, schema), applies the runbook step.	One automated restart; repeat failure goes to a human.

C. Loading into Snowflake (Snowpipe Streaming, Snowpipe, GCS)

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
C1	**Duplicates after a connector restart.** Offset tokens reset or channels are reopened, so rows load twice.	DE	**DQ** checks Bronze duplicate rate per channel after every restart event and reports it; Silver MERGE dedupes by key.	Silver keeps the dedupe; the agent only reports.
C2	**Batch loads fail silently.** GCS file lands, COPY rejects rows, nobody notices. Or the Pub/Sub notification integration is misconfigured and nothing loads.	DE	**WATCH** compares files in the bucket with COPY_HISTORY and SYSTEM$PIPE_STATUS; reports files never loaded or partially loaded.	Reload uses a named file list, never a full re-scan.
C3	**Load-history dedupe.** Re-uploading a fixed file with the same name is skipped by Snowpipe.	DE	**WATCH** recognises the “already loaded” case and proposes the correct reload path.	FORCE reloads require approval (risk of duplicates).
C4	**Backfill overlaps streaming.** The boundary minute exists in both paths.	DE	**DQ** checks the overlap window after each backfill and confirms the MERGE kept one row per key.	Overlap window defined explicitly per backfill run.
C5	**Small-file cost.** Thousands of tiny files raise Snowpipe overhead.	DE	**GUARD** reports cost per loaded GB and suggests batching thresholds.	Recommendation only.

D. Bronze to Silver (Streams + Tasks, Dynamic Tables)

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
D1	**Stale stream.** A stream not consumed within the retention window becomes stale; the only fix is recreate, and changes in between are lost.	DE	**DOCTOR** watches stream staleness dates and warns days ahead; if stale, runs the recreate + gap-backfill runbook.	Recreate needs approval; the gap backfill is logged.
D2	**Suspended or failing tasks.** A task fails repeatedly and auto-suspends; Silver stops updating.	DE	**DOCTOR** reads TASK_HISTORY errors, classifies (SQL error, warehouse, permissions), fixes known causes, resumes.	RESUME TASK allowed only for classified known causes.
D3	**Non-deterministic MERGE.** Duplicate keys in the source batch make MERGE fail.	DE	**DOCTOR** identifies the duplicate keys, explains which path produced them (see C1/C4).	Fix goes into the QUALIFY dedupe, via PR.
D4	**Silent nulls from TRY_TO_ casts.** Bad values become NULL instead of failing; data quietly disappears from metrics.	DE + Analyst	**DQ** tracks the NULL rate per cast column per hour and alerts on jumps.	Threshold reviewed by a human once, then fixed in config.
D5	**Dynamic Table lag misses.** Refreshes fall behind TARGET_LAG or fail.	DE	**DOCTOR** reads DYNAMIC_TABLE_REFRESH_HISTORY and explains why (upstream, warehouse size, query change).	Warehouse resizing needs approval.

E. Silver to Gold (dbt)

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
E1	**Full rebuilds get slow and expensive.** Table materialisation rebuilds all history every run.	DE	**XFORM** drafts incremental versions with a late-arrival lookback window and compares outputs against the full rebuild.	Merge only if row-by-row diff is zero on a test window.
E2	**2 a.m. test failure.** dbt test fails; the on-call engineer has to work out which test, which rows, and whether it is real.	DE	**DOCTOR** reads run_results.json, runs the failing test's SQL, returns failing rows and the likely cause.	The agent never disables or weakens a test.
E3	**Upstream schema change breaks models.**	DE	**XFORM** diffs INFORMATION_SCHEMA before/after, finds affected models via the dbt DAG, opens a fix PR.	CI must pass; human review required.
E4	**Grants and policies lost on rebuild.** CREATE OR REPLACE drops row access policies and table grants (found and fixed in this repo in commit a480e77).	DE	**GUARD** runs the POLICY_REFERENCES check after every dbt run; missing policy is a sev-1.	Sev-1 pauses agent answers until fixed.
E5	**Window metrics wrong at boundaries.** Rolling volatility runs across the overnight gap or across a split.	Analyst	**DQ** tests that rolling windows reset per session; **XFORM** proposes partitioning by trading date.	Metric definition change requires CURATOR update too.

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
E6	**Docs drift.** README run order no longer matches the code (also found in this repo).	DE	**XFORM** checks documented commands and object names against the code on every PR.	Flags only; a human edits the docs.

F. Semantic layer, Cortex Analyst, Cortex Agent (analyst-facing)

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
F1	**Ambiguous questions.** “What's AAPL's return today?” before the open, or “return” meaning close-to-close vs open-to-close.	Analyst	**ANALYST** asks one clarifying question or states the assumption it used (definition, timezone, session).	Definitions come from the semantic model, not the model's guess.
F2	**Wrong SQL that looks right.** Bad join, wrong grain, wrong filter: a confident wrong number.	Analyst	**ANALYST** shows the SQL it ran; **CURATOR** regression-tests every verified query nightly against sql/08_validation twins.	Answers without a verified pattern are labelled as such.
F3	**Two sources of truth.** semantic_model.yaml and semantic_view.sql define metrics separately and drift apart.	DE + Analyst	**CURATOR** diffs the two definitions on every PR and fails CI on mismatch.	Single owner signs off on metric changes.
F4	**Routing errors.** Agent sends a “why did it move” question to SQL only, or a numeric question to search.	Analyst	**CURATOR** keeps a labelled set of routing examples and scores the agent on it after each instruction change.	Routing accuracy target agreed before rollout.
F5	**RAG problems.** News index lags (TARGET_LAG 1 h), wrong-ticker articles (“F” is Ford; “Apple” in a recipe), missing citations.	Analyst	**ANALYST** filters search by symbol attribute, always cites, and states article timestamps. **DQ** measures retrieval precision on a labelled set.	No citation means no news claim in the answer.
F6	**“The data is wrong” that is actually the 15-minute delay.** Delayed-entitlement users see older prices than a live screen.	Analyst	**ANALYST** states the as-of timestamp and entitlement in every price answer.	Policy enforced in Snowflake; the agent only explains it.
F7	**Suspect bars in aggregates.** One bad print distorts daily return or volatility.	Analyst	**ANALYST** checks is_suspect rows in scope and mentions them; offers the figure with and without them.	Suspect rows are flagged, never silently dropped.
F8	**Waiting on engineering for every new metric.**	Analyst	**CURATOR** mines unanswered questions from the audit log and drafts new metrics + verified queries as a PR.	Data engineer reviews every new metric.
F9	**Slow or expensive answers.** Agent calls time out or cost too much per question.	Analyst + DE	**GUARD** tracks latency and credits per question type; **CURATOR** adds pre-aggregated views for the heaviest patterns.	Per-role statement timeout already in sql/09_guardrails.

G. Governance, security and cost

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
G1	**Licence leakage.** A role that shouldn't see real-time prices gets them through a new grant or a new table.	DE	**GUARD** diffs grants daily and checks every new Gold table for the row access policy.	New grants to REALTIME_DESK always need human approval.
G2	**Prompt injection through news.** A news article contains text like “ignore your instructions and…”, and the agent reads it as a command.	DE	**GUARD** screens retrieved text and the agent treats search results as data only; the agent's role has read-only grants, so even a successful injection cannot change data.	Least-privilege role is the real defence, not the screen.
G3	**Runaway spend.** A loop of agent queries or a bad dbt model burns credits overnight.	DE	**GUARD** watches metering per warehouse and per role and suspends at the resource-monitor threshold.	Resource monitor in sql/09_guardrails is the hard stop.
G4	**Audit gaps.** Can't prove who asked what and what data they saw.	DE	**GUARD** reconciles AGENT_QUERY_AUDIT_LOG with QUERY_HISTORY and reports missing entries.	Audit table is append-only.
G5	**Secret rotation.** Alpaca keys and Snowflake key pairs expire or leak.	DE	**GUARD** tracks key age and reminds before expiry.	Agents never read or store secret values.

I. Data volume and overload

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
I1	**Market-open and close spikes.** Volume at 9:30 and 16:00 ET is many times the midday rate; producer, Kafka and Snowpipe Streaming all lag at once.	DE	**SCALE** pre-scales connector tasks and warehouses before the open using the market calendar, then scales back after the spike.	Schedule pre-approved; hard size cap.
I2	**Event-day surges.** CPI, FOMC, earnings or a meme-stock day brings sudden, unplanned volume far above normal.	DE	**SCALE** compares throughput with the same time-of-day baseline, scales within limits, and warns **DQ** to expect late data.	Resource-monitor credit ceiling still applies.
I3	**Producer backpressure.** The producer's send buffer fills; it either drops messages or runs out of memory.	DE	**WATCH** tracks buffer use and send errors and alerts before the buffer is full.	Producer configured to block, never silently drop.
I4	**Kafka broker disk full.** A surge plus long retention fills disks; brokers stop accepting writes.	DE	**SCALE** projects time-to-full from the fill rate and proposes retention or disk changes early.	Retention/disk changes need approval.
I5	**Snowpipe Streaming throttling.** Client hits throughput or channel limits and starts returning errors.	DE	**WATCH** reads client errors; **SCALE** proposes more channels or connector tasks.	Changes go through config review.
I6	**Warehouse queuing.** Tasks, dbt and agent queries compete for compute; queries queue and Silver falls behind at peak.	DE	**SCALE** reads queued time in QUERY_HISTORY and proposes separate or multi-cluster warehouses (ANALYST_WH is already separate).	Bigger warehouses need approval.
I7	**Many analysts at once.** Morning question bursts hit Cortex and the warehouse together; answers time out.	Analyst	**GUARD** rate-limits per user; **SCALE** enables multi-cluster scaling for ANALYST_WH within limits.	Per-role statement timeout stays on.
I8	**Huge scans.** A broad question makes the agent scan years of trades.	Analyst + DE	**ANALYST** adds a default date range and says so; **SCALE** suggests clustering keys for the common filters.	Statement timeout is the hard stop.
I9	**Storage growth from churn.** MERGE every minute multiplies Time Travel and Fail-safe storage.	DE	**GUARD** reports storage per table including Time Travel bytes and proposes transient Bronze or shorter retention.	Retention changes need approval.

J. Outages and recovery (server down)

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
J1	**Alpaca outage or degraded feed.** Vendor is down or sending partial data during market hours.	DE + Analyst	**WATCH** detects it, records the window in an OPS outage table; **ANALYST** tells users that window is incomplete; backfill runs when the vendor recovers.	Outage windows are recorded, never hidden.
J2	**Kafka broker down or quorum lost.** Producer can't write.	DE	**WATCH** detects write failures; **TRIAGE** pages with the broker state and the data at risk.	Agents never restart cluster nodes on their own.
J3	**Kafka Connect worker crash or rebalance storm.** Tasks keep moving between workers and nothing loads.	DE	**WATCH** spots repeated rebalances in the Connect logs and recommends the runbook fix.	One automated restart, then a human.
J4	**Snowflake unavailable.** (region incident, or a warehouse can't resume).	DE	**TRIAGE** confirms with a probe query and the status page, and works out how long Kafka retention can buffer (see B4). Failover to a second region only if replication is set up.	Failover is always a human decision.
J5	**Cortex unavailable or model not available in the region.** Agent questions fail.	Analyst	**ANALYST** falls back to verified-query results or the direct Cortex Analyst path and says it is in degraded mode.	No made-up answers in degraded mode.
J6	**GCS or Pub/Sub notification outage.** Files land but no notification arrives, so nothing loads.	DE	**WATCH** compares the bucket listing with COPY_HISTORY and proposes ALTER PIPE ... REFRESH for the missed window.	Refresh scoped to the window; approval needed.
J7	**Recovery storm.** After an outage everything catches up at once and causes a second overload.	DE	**SCALE** paces the catch-up; **DQ** confirms completeness for the outage window afterwards.	Catch-up plan logged with the incident.

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
J8	**Partial outage.** Only some symbols or one partition stop; the global “last message” metric still looks fine.	DE	**DQ** checks freshness per symbol, not just overall.	Per-symbol thresholds account for illiquid names.
J9	**Bad deploy takes the pipeline down.** A SQL or dbt change breaks tasks or corrupts a table.	DE	**DOCTOR** links the failure to the change (git history + QUERY_HISTORY) and proposes a rollback or a Time Travel restore (AT/BEFORE, UNDROP).	Restores need approval.

K. Time and timestamp matching

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
K1	**Event time vs load time.** trade_ts (when it happened) and _loaded_at (when it arrived) get mixed up; metrics use the wrong one.	DE + Analyst	**DQ** tracks the lateness distribution (load minus event time); **XFORM** checks every model aggregates on event time.	Event-time rule written in AGENTS.md.
K2	**Producer clock skew.** A host without NTP stamps wrong times; latency comes out negative or impossible.	DE	**WATCH** flags negative or implausible latencies and names the host.	Fix is NTP on the host, not adjusting data.
K3	**Bar timestamp convention.** Alpaca stamps a minute bar at the start of the minute; other sources use the end. Joins are off by one minute.	DE + Analyst	**DQ** asserts the convention on each source; **CURATOR** documents it in the semantic model.	Convention converted at Silver, once.
K4	**Trades don't match bars.** Bars rebuilt from trades differ from vendor bars (feed coverage, condition filters, boundary inclusive/exclusive).	DE + Analyst	**DQ** reconciles rebuilt vs vendor bars within a tolerance and explains each difference type.	Tolerances agreed with analysts.
K5	**Late data after aggregates are built.** A bar is final, then a correction arrives.	DE	**DQ** measures how late data really arrives so the incremental lookback window (E1) is sized from evidence.	Lookback set in config, reviewed.
K6	**Out-of-order arrival.** Messages from different partitions arrive out of sequence; logic that assumes arrival order breaks.	DE	**XFORM** checks every window function orders by event time with a tie-breaker (trade_id).	Enforced by a dbt test.
K7	**Precision truncation.** Nanosecond timestamps cut to milliseconds or seconds create duplicate (symbol, ts) keys and MERGE collisions.	DE	**DQ** checks key uniqueness and timestamp precision per column.	Columns declared TIMESTAMP_NTZ(9).
K8	**Session timezone vs NTZ.** CURRENT_TIMESTAMP() is TIMESTAMP_LTZ; comparing it with a UTC NTZ column depends on the session TIMEZONE. With Snowflake's default America/Los_Angeles, a 15-minute delay rule can hide hours of data. Found in this repo's DELAYED_DATA_POLICY and load-time defaults; fixed to use SYSDATE().	DE + Analyst	**GUARD** tests the policy under several session timezones and fails if the visible window changes.	Compare against a UTC NTZ value (SYSDATE()); sql/08 checks the delay under two session timezones.
K9	**News-to-price alignment.** Using updated_at instead of published_at, or the wrong window, pins the wrong article to a price move.	Analyst	**ANALYST** searches only news published shortly before the move and states the time gap.	No causal claim without a timestamp.
K10	**Trading-day boundary.** date_key is the UTC day, so after-hours bars past 8 p.m. ET fall on the next day.	Analyst	**DQ** flags bars whose UTC date differs from the exchange date; **XFORM** proposes an exchange-time date_key.	Metric change also updates CURATOR.

L. Market structure and data correctness

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
L1	**Trading halts and LULD pauses.** No trades for minutes looks exactly like an outage.	DE + Analyst	**DQ** checks halt news before raising a gap; **WATCH** suppresses the outage alert for that symbol.	Halt windows stored with source.

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
L2	**Listings, delistings, symbol changes.** dim_symbols goes stale; the agent says “no data” for a valid ticker.	DE + Analyst	**DQ** diffs the symbol master daily and reports adds, removals and renames.	History kept for inactive symbols.
L3	**Bad prints the 10% rule misses.** Outliers within 10%, or legitimate moves above it.	DE + Analyst	**DQ** scores bars with the ONNX anomaly model alongside is_suspect and reports disagreements.	Flag only; never delete.
L4	**Illiquid gaps.** Thin symbols have minutes with no trades; teams disagree on forward-fill.	Analyst	**CURATOR** documents one gap rule; **XFORM** implements it consistently.	Rule visible in metric descriptions.
L5	**Extended hours mixed with the regular session.** Pre- and after-hours distort volume and returns.	Analyst	**ANALYST** defaults to the regular session and says so; session is a semantic-model dimension.	Default stated in every answer.
L6	**Unit mismatch.** A new source reports cents or lots instead of dollars or shares.	DE	**DQ** range-checks price and size per symbol against history.	New sources start in quarantine.

M. Concurrency, reprocessing and change

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
M1	**Task overlap.** A MERGE takes longer than its schedule; runs are skipped and lag builds.	DE	**DOCTOR** compares task duration with the schedule and recommends a new interval or warehouse size.	ALLOW_OVERLAPPING_EXECUTION stays false.
M2	**Lock contention.** A backfill MERGE and the streaming MERGE hit the same table; lock waits and timeouts follow.	DE	**DOCTOR** finds blocked queries and schedules backfills outside streaming peaks.	Backfill windows planned in advance.
M3	**Reprocessing history.** A bug fix means rebuilding months of data without double counting.	DE	**DOCTOR** plans the rebuild in a zero-copy clone, compares results, then proposes a swap.	Swap needs approval; old table kept via Time Travel.
M4	**Real-time vs batch disagree.** The streaming path and the backfill give different numbers for the same day.	DE + Analyst	**DQ** reconciles them daily; after T+1 the batch path is the source of truth.	Rule documented for analysts.
M5	**Environment drift.** Dev and prod objects differ, so a change works in dev and fails in prod.	DE	**XFORM** diffs GET_DDL output between environments before deploys.	Deploys blocked on drift.
M6	**Bad data reaches Gold.** Wrong numbers are already published.	DE + Analyst	**DOCTOR** proposes a Time Travel restore; **ANALYST** flags answers given during the bad window (from the audit log).	Restore needs approval; users told.

N. Analyst trust and reproducibility

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
N1	**Same question, different answer.** Asked at 10:05 and 10:30, numbers differ because of late data and the delay window.	Analyst	**ANALYST** gives an as-of time with every answer; the audit log keeps the SQL so it can be re-run with Time Travel AT that time.	Audit log is append-only.
N2	**Look-ahead bias.** Backtests use corrected prints or news that weren't available at the time.	Analyst	**CURATOR** provides point-in-time views filtered on _loaded_at.	Backtest questions use point-in-time views.
N3	**Survivorship bias.** Delisted symbols missing from history inflate returns.	Analyst	**ANALYST** includes inactive symbols for historical questions and warns when the universe changed.	dim_symbols keeps inactive rows.
N4	**Metric disputes.** Teams calculate VWAP or return differently.	Analyst	**CURATOR** keeps one definition per metric, with synonyms that map to it.	Metric owner signs off.
N5	**Dashboard load.** BI tools refresh Gold every few seconds.	DE	**SCALE** routes BI to its own warehouse and relies on the result cache; **GUARD** reports top consumers.	Recommendation only.
N6	**Questions outside the data.** Fundamentals, options, other markets: the risk is a made-up answer.	Analyst	**ANALYST** says the data isn't available; **CURATOR** logs it as demand for new data.	No answer without a source.

P. SQL correctness in Snowflake

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
P1	**Join fan-out inflates totals.** Joining bars to a table with several rows per symbol (news, for example) repeats each bar, so SUM(volume) is multiplied. *(SQL example in the patterns section)*	Analyst + DE	**SQLCOACH** checks the grain of both sides of every join and compares row counts before and after the join; it rewrites to aggregate first or use EXISTS.	Rewrite must return the same rows as the reference query on test data.
P2	**DISTINCT used to hide duplicates.** A bad join creates duplicates and SELECT DISTINCT is added to make them go away; the root cause stays and counts are still wrong elsewhere. *(SQL example in the patterns section)*	Analyst + DE	**SQLCOACH** flags DISTINCT over joined results, finds the join that duplicates rows, and fixes the join or dedupes explicitly with QUALIFY.	DISTINCT is accepted only with a comment explaining why.
P3	**NOT IN with a NULL returns nothing.** If the subquery contains a single NULL, NOT IN is never true and the query silently returns zero rows. *(SQL example in the patterns section)*	Analyst + DE	**SQLCOACH** rewrites NOT IN (subquery) as NOT EXISTS whenever the subquery column is nullable.	Rule applied automatically in review; no exceptions.
P4	**NULLs in aggregates and filters.** AVG ignores NULLs, COUNT(col) skips them while COUNT(*) does not, and col <> 'X' silently drops NULL rows. *(SQL example in the patterns section)*	Analyst	**SQLCOACH** flags aggregates and inequality filters on nullable columns and asks for an explicit choice; **ANALYST** states how NULLs were treated.	Choice documented in the semantic model metric.
P5	**FLOAT used for money.** Prices or notional summed as FLOAT drift by fractions of a cent and equality checks fail. *(SQL example in the patterns section)*	DE	**SQLCOACH** flags FLOAT/DOUBLE on price, notional or vwap columns and casts; **XFORM** changes the column to NUMBER with fixed scale.	Column type changes go through RELEASE.
P6	**LIMIT or MAX_BY without a deterministic order.** LIMIT without ORDER BY, or ORDER BY with ties, returns a different 'top 10' each run; MAX_BY picks an arbitrary row on ties. *(SQL example in the patterns section)*	Analyst	**SQLCOACH** requires ORDER BY with a unique tie-breaker for every LIMIT, and a unique ordering key for MAX_BY/MIN_BY.	Applies to verified queries too.
P7	**DATEDIFF counts boundaries, not elapsed time.** DATEDIFF('day', '23:59', '00:01 next day') is 1 although two minutes passed; durations come out wrong. *(SQL example in the patterns section)*	Analyst + DE	**SQLCOACH** flags DATEDIFF at day/hour granularity used as a duration and rewrites using seconds or minutes.	Boundary counting is allowed when the question is about calendar days.
P8	**Week boundaries depend on session settings.** WEEK_START and WEEK_OF_YEAR_POLICY change DATE_TRUNC('week') and WEEK(); two users get different weekly numbers. *(SQL example in the patterns section)*	Analyst	**SQLCOACH** flags session-dependent week functions; **CURATOR** adds one ISO-week column to DIM_DATE so the definition lives in one place.	Weekly metrics defined once in the semantic model.
P9	**Quoted identifiers are case-sensitive.** A table created with "Symbol" in quotes (often by pandas write_pandas) can't be queried as symbol; queries fail with 'invalid identifier'. *(SQL example in the patterns section)*	DE + Analyst	**SQLCOACH** detects mixed-case quoted columns in INFORMATION_SCHEMA and recommends unquoted upper-case names (quote_identifiers=False when loading).	Naming convention enforced in RELEASE checks.
P10	**Semi-structured (VARIANT) traps.** raw:price is still VARIANT, so comparisons and sums behave unexpectedly; FLATTEN drops rows with empty arrays unless OUTER => TRUE. *(SQL example in the patterns section)*	DE	**SQLCOACH** requires an explicit ::type cast on every VARIANT path and OUTER => TRUE where empty arrays must be kept.	Casting rules applied at Silver.
P11	**Division by zero.** One zero denominator (an open price of 0 in a bad print) fails the whole query, or returns infinity downstream. *(SQL example in the patterns section)*	Analyst + DE	**SQLCOACH** rewrites divisions on columns that can be zero with DIV0NULL or NULLIF.	Rows that hit the rule are counted and reported, not hidden.

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
P12	**UPDATE/MERGE with duplicate source rows.** UPDATE ... FROM a source with duplicate keys picks an arbitrary row silently (MERGE errors, UPDATE doesn't by default). *(SQL example in the patterns section)*	DE	**SQLCOACH** requires the source of every UPDATE/MERGE to be unique on the join key (QUALIFY ROW_NUMBER() = 1); **DOCTOR** handles the runtime error case (D3).	ERROR_ON_NONDETERMINISTIC_UPDATE set to TRUE for pipeline roles.
P13	**String comparison surprises.** Comparisons are case-sensitive ('aapl' is not 'AAPL'), trailing spaces don't match, and numbers stored as text sort as text ('100' < '20'). *(SQL example in the patterns section)*	Analyst + DE	**SQLCOACH** flags comparisons on raw text columns; **DQ** checks that Silver stores symbols upper-cased and trimmed and numbers as numbers.	Standardisation happens once, in Silver.
P14	**Window frames and partitions.** A running total with ORDER BY uses a RANGE frame and includes ties; LAG without PARTITION BY symbol reads the previous symbol's price. *(SQL example in the patterns section)*	Analyst + DE	**SQLCOACH** requires PARTITION BY on per-symbol windows and an explicit ROWS frame; **XFORM** adds a dbt test for the pattern.	Enforced by a dbt test (see K6, E5).

Q. Performance and query tuning

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
Q1	**Spilling to disk.** Large sorts or joins don't fit in warehouse memory and spill to local or remote storage; the query runs many times slower.	DE + Analyst	**SQLCOACH** reads bytes_spilled_* in QUERY_HISTORY, filters and aggregates earlier in the query; **SCALE** suggests a larger warehouse only for that workload.	Size changes need approval.
Q2	**No partition pruning.** Wrapping the filter column in a function or casting it makes Snowflake scan every micro-partition. *(SQL example in the patterns section)*	DE + Analyst	**SQLCOACH** compares partitions_scanned with partitions_total and rewrites filters as ranges on the raw column.	Rewrite must return identical results.
Q3	**Clustering costs more than it saves.** Automatic clustering on a table updated every minute keeps re-clustering and burns credits.	DE	**SQLCOACH** checks SYSTEM$CLUSTERING_INFORMATION against the common filters; **GUARD** compares AUTOMATIC_CLUSTERING_HISTORY credits with query savings.	Clustering keys changed only through RELEASE.
Q4	**Exploding or cartesian joins.** A missing or wrong join condition produces billions of rows and a query that never finishes. *(SQL example in the patterns section)*	Analyst + DE	**SQLCOACH** reads GET_QUERY_OPERATOR_STATS, finds the join whose output rows far exceed its inputs, and names the missing condition.	Statement timeout stops the runaway first.
Q5	**Slow UDFs.** A row-by-row Python or JavaScript UDF does what a built-in SQL function could do.	DE	**SQLCOACH** suggests the built-in equivalent or a vectorised UDF and measures the difference on a clone.	Recommendation only.
Q6	**Wrong warehouse size for the workload.** Too large and mostly idle, or too small and queuing and spilling.	DE	**SCALE** reads WAREHOUSE_LOAD_HISTORY per warehouse and recommends a size and auto-suspend per workload; **GUARD** shows the cost effect.	Changes need approval.
Q7	**Slow first queries after resume.** After auto-suspend the local cache is empty, so the first dashboard of the morning is slow and users think the system is broken.	Analyst	**ANALYST** explains a cold start when it happens; **SCALE** tunes auto-suspend for BI hours.	Longer auto-suspend weighed against idle cost.
Q8	**Cached results mislead tuning.** The result cache returns a repeated query instantly, so 'my rewrite is faster' comparisons are wrong.	DE	**SQLCOACH** disables USE_CACHED_RESULT when benchmarking and reports both runs.	Benchmark method recorded with the result.
Q9	**Reading columns you don't need.** SELECT * on wide tables reads every column; Snowflake is columnar, so unused columns still cost time and credits. *(SQL example in the patterns section)*	Analyst + DE	**SQLCOACH** replaces SELECT * with the columns actually used downstream.	SELECT * allowed in ad hoc exploration with LIMIT.

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
Q10	**Thousands of tiny queries.** An app loops over symbols and sends one query each; overhead dominates and the warehouse never suspends. *(SQL example in the patterns section)*	DE	**SQLCOACH** groups identical parameterised queries in QUERY_HISTORY and proposes one set-based query.	Fix lives in application code, reviewed by RELEASE.
Q11	**Nobody can read the query profile.** People can see a query is slow but not where the time goes.	DE + Analyst	**SQLCOACH** summarises GET_QUERY_OPERATOR_STATS in plain words, for example: 78% of time in one join, 4 GB spilled, 2% of partitions pruned.	Summary links to the raw profile.

R. Development to production

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
R1	**No separate environments.** Changes are made directly in production.	DE	**RELEASE** sets up DEV, TEST and PROD databases (TEST as a zero-copy clone of PROD) and checks that only the deploy role can change PROD.	Direct PROD DDL by people is blocked by role.
R2	**Untracked manual changes.** Objects are edited in Snowsight and never make it into git.	DE	**RELEASE** keeps every change as a versioned migration (for example schemachange or Terraform) and diffs GET_DDL against the repo daily.	Drift opens a ticket; it is never auto-reverted.
R3	**Scripts that can't be re-run.** CREATE without IF NOT EXISTS or a plain INSERT fails or duplicates data on the second run.	DE	**SQLCOACH** flags non-idempotent statements in review; **RELEASE** runs every migration twice in CI to prove it.	Idempotent DDL is a repo convention (AGENTS.md).
R4	**CI rebuilds everything.** dbt CI runs all models on every pull request; reviews take an hour.	DE	**RELEASE** runs only changed models and their children (state:modified+) and defers the rest to the production manifest.	Full build still runs nightly.
R5	**Tests pass on tiny dev data.** Dev has a few rows; production fails on volume, duplicates or edge cases.	DE	**RELEASE** runs CI against a zero-copy clone of production so tests see real data at no storage cost.	Clones dropped after the run.
R6	**Breaking changes for consumers.** Renaming or dropping a column breaks dashboards, saved worksheets and the agent's semantic model.	DE + Analyst	**RELEASE** runs impact analysis (dbt lineage, OBJECT_DEPENDENCIES, ACCESS_HISTORY for who read the column in 30 days, semantic model references); **CURATOR** updates the semantic model in the same change.	Breaking changes need a deprecation period.
R7	**No data contracts.** An upstream team changes a type or drops a column and downstream models break.	DE	**RELEASE** turns on dbt model contracts for Gold; **XFORM** keeps the contract in sync with the model.	Contract changes need both owners.
R8	**Rolling back a release.** Code rolls back easily; data written by the bad version does not.	DE	**RELEASE** plans each release with a rollback: a clone of affected tables before deploy and ALTER TABLE ... SWAP WITH to restore; **DOCTOR** runs it if needed.	Swap needs approval.
R9	**Deploy order wrong.** Objects depend on each other; deploying in the wrong order fails (this repo's README run-order bug).	DE	**RELEASE** builds the dependency order and proves it by deploying to an empty database in CI.	A deploy that fails on an empty database is blocked.
R10	**Secrets in code.** Passwords or keys committed to git or pasted into SQL files.	DE	**RELEASE** runs secret scanning on every pull request; **GUARD** rotates anything found.	A found secret is treated as leaked.
R11	**Deploying while tasks are writing.** A table changes while a task is mid-MERGE; the task fails or writes to the old shape.	DE	**RELEASE** suspends the affected tasks, deploys, then resumes them in dependency order; **DOCTOR** verifies they caught up.	Deploy plan lists every task touched.
R12	**Unknown lineage.** 'What feeds this number?' takes a day to answer.	DE + Analyst	**RELEASE** and **CURATOR** answer from dbt docs, OBJECT_DEPENDENCIES and ACCESS_HISTORY, from source to semantic metric.	Lineage regenerated on every deploy.

S. Security and access

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
S1	**Role sprawl.** Hundreds of roles and grants; nobody knows who can see what.	DE	**GUARD** maps the role hierarchy, finds grants unused for 90 days in ACCESS_HISTORY, and proposes removals.	Removals need the data owner's approval.
S2	**Objects owned by people who left.** Tables and tasks owned by a personal role can't be changed once that person leaves.	DE	**GUARD** finds objects owned by personal roles and proposes GRANT OWNERSHIP ... COPY CURRENT GRANTS to a functional role.	Ownership transfer needs approval.
S3	**Owners grant access around the rules.** Whoever owns a table can grant it to anyone, bypassing central access control.	DE	**GUARD** recommends WITH MANAGED ACCESS on GOLD so only the security role grants access.	Schema change goes through RELEASE.
S4	**ACCOUNTADMIN used for daily work.** People run everyday queries with the most powerful role.	DE	**GUARD** reports queries run as ACCOUNTADMIN or SECURITYADMIN and who ran them.	Report only; humans follow up.
S5	**Service accounts with passwords.** Connectors and apps log in with a username and password.	DE	**GUARD** lists users with password authentication and recommends TYPE = SERVICE with key-pair or OAuth, plus a rotation schedule.	Agents never handle secret values.
S6	**No network policy.** The account accepts logins from anywhere on the internet.	DE	**GUARD** checks for account and user network policies and drafts one from LOGIN_HISTORY source IPs.	Applying a policy needs approval (it can lock people out).
S7	**Human users without MFA or SSO.** Password-only logins for people.	DE	**GUARD** reports human logins without a second factor from LOGIN_HISTORY.	Report only.
S8	**Sensitive data not tagged.** New columns with personal or sensitive text (for example questions stored in the audit log) have no tags or masking.	DE	**GUARD** runs Snowflake data classification on new tables and proposes tags and masking policies.	Tags and policies applied through RELEASE.
S9	**Agent role has more access than it needs.** ANALYST_AGENT is granted Silver or Bronze, so it can bypass the semantic layer and the delay policy's intent.	DE	**GUARD** checks ANALYST_AGENT can only read Gold, the semantic view and the search service, and alerts on anything else.	Any new agent grant needs approval.

T. Cost

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
T1	**No cost attribution.** Nobody can say which team, agent or dashboard spends the credits.	DE	**GUARD** requires a QUERY_TAG per app and agent and reports spend from QUERY_ATTRIBUTION_HISTORY by tag, user and warehouse.	Untagged queries are reported.
T2	**Idle warehouses.** Auto-suspend set to 10+ minutes, or never, on warehouses used in short bursts.	DE	**GUARD** flags long auto-suspend on interactive warehouses; **SCALE** recommends a value from the query pattern.	Changes need approval.
T3	**Hidden serverless costs.** Snowpipe, Snowpipe Streaming, serverless tasks, Dynamic Table refreshes, search optimisation, automatic clustering, Cortex Search and Cortex models bill outside warehouses.	DE	**GUARD** reports METERING_HISTORY by service type every day, with week-over-week change.	Spikes over a threshold page the owner.
T4	**Expensive dev copies.** Full copies instead of zero-copy clones, or long Time Travel on dev databases.	DE	**GUARD** flags dev tables that aren't clones and dev databases with long retention; **RELEASE** uses clones by default.	Recommendation only.
T5	**Schedules faster than the data.** A task runs every minute when data arrives every five; a Dynamic Table's TARGET_LAG is tighter than anyone needs.	DE	**GUARD** compares data arrival rate with refresh frequency; **DOCTOR** proposes the slower schedule.	Freshness requirement agreed with analysts first.
T6	**The same heavy query, again and again.** Many dashboards run the same expensive query.	DE + Analyst	**GUARD** finds repeated heavy queries; **CURATOR** proposes a materialised table or view the semantic model can use.	New objects go through RELEASE.

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
T7	**No budget, surprise bill.** Spend is noticed only when the invoice arrives.	DE	**GUARD** sets budgets and resource monitors per warehouse and per agent, with alerts at 50/80/100%.	The resource monitor in sql/09_guardrails is the hard stop.
T8	**Unknown cost per agent question.** LLM and Cortex Analyst costs per question aren't tracked.	DE + Analyst	**GUARD** joins Cortex usage history with the agent audit log to report cost per question type and per user.	Per-user daily limits if needed.

U. Analyst self-service

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
U1	**Can't find the right table.** RAW_BARS, SILVER.BARS, FCT_BARS and SYMBOLS_DT all look like 'the bars table'.	Analyst	**CURATOR** tags certified Gold objects and writes 'use this, not that' notes; **ANALYST** answers 'where is X' questions from them.	Only Gold is certified for analysis.
U2	**Columns without descriptions.** Nobody knows what bar_return or is_suspect means exactly.	Analyst	**CURATOR** fills column COMMENTs from dbt schema.yml and the semantic model, so the same text appears everywhere.	Descriptions reviewed with the metric owner.
U3	**BI connections time out.** Tableau, Power BI or Excel extracts hit timeouts or row limits.	Analyst	**ANALYST** suggests an aggregated query or view; **SCALE** gives BI its own warehouse.	Statement timeout stays in place.
U4	**Millions of rows into Excel.** Analysts try to export far more rows than a spreadsheet can hold.	Analyst	**ANALYST** proposes aggregation first, or an unload to a stage (COPY INTO @stage) for genuinely large extracts.	Unloads follow data-sharing rules (feed licence).
U5	**Saved queries break after a change.** Worksheets and dashboards reference a renamed column.	Analyst	**RELEASE**'s impact analysis (R6) lists affected users from ACCESS_HISTORY; **CURATOR** notifies them before the change ships.	Deprecation period for breaking changes.
U6	**Numbers don't match another system.** Finance or a vendor terminal shows a different figure.	Analyst	**DQ** and **ANALYST** produce a reconciliation that names each difference: definition, cut-off time, timezone, feed coverage or scope.	Reconciliations are saved and reused.
U7	**Copy-pasting old SQL.** Analysts copy old queries, bugs included.	Analyst	**CURATOR** keeps a library of verified queries; **SQLCOACH** reviews new queries against the patterns in section P.	Library entries have owners.
U8	**'Does not exist or not authorized'.** Analysts get a permission error with no explanation.	Analyst	**ANALYST** explains which role grants access and how to request it; **GUARD** routes the request to the data owner.	Agents never grant access themselves.
U9	**Working from stale extracts.** Analysts download data, work offline, and report outdated numbers.	Analyst	**ANALYST** encourages live queries and stamps every answer with an as-of time.	As-of time on every answer.

O. Operations and people

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
O1	**Root cause spans six systems.** A stale Gold number could start in Alpaca, Kafka, Snowpipe, a task, dbt or the agent.	DE	**TRIAGE** walks the chain upstream, asking each specialist agent for its health, and returns the first broken link with evidence.	Summary goes to a human; actions stay with specialists.
O2	**Reconciliation across layers.** Kafka offsets vs Bronze rows vs Silver vs Gold don't add up.	DE	**DQ** runs per-hour count reconciliation and reports the layer where rows disappear.	Tolerances set per layer (dedupe expected in Silver).
O3	**Alert fatigue.** Dozens of alerts for one root cause.	DE	**TRIAGE** groups related alerts into one incident with a timeline.	Raw alerts still logged.
O4	**Tribal knowledge and missing runbooks.** Only one person knows how to fix a stale stream.	DE	Every agent fix is written as a runbook step first; **TRIAGE** writes a post-incident note proposing a new runbook entry.	Runbooks reviewed and versioned in git.
O5	**No CI, no tests (this repo's current gap).** Changes reach Snowflake untested.	DE	**XFORM** runs dbt parse/test and Python tests on every PR once CI exists.	Build pytest + CI first (README Target items).

SQL patterns the agents learn (18)

Each pair is a real mistake and its fix, written against this repo's tables where possible. SQLCOACH uses them as reference knowledge when reviewing SQL, and each pair is also an eval case: the agent gets the bad query and passes only if it names the right problem and its rewrite returns exactly what the fixed query returns on seeded test data.

#	Problem	Mistake	Fix
P1	Join fan-out inflates totals	`SELECT b.symbol, SUM(b.volume) AS volume FROM GOLD.FCT_BARS b JOIN SILVER.NEWS n ON n.symbols ILIKE '%' || b.symbol || '%' GROUP BY b.symbol;`	`SELECT b.symbol, SUM(b.volume) AS volume FROM GOLD.FCT_BARS b WHERE EXISTS (SELECT 1 FROM SILVER.NEWS n WHERE n.symbols ILIKE '%' || b.symbol || '%') GROUP BY b.symbol;`
P2	DISTINCT used to hide duplicates	`SELECT DISTINCT t.trade_id, t.price, t.size FROM SILVER.TRADES t JOIN BRONZE.RAW_TRADES r ON r.trade_id = t.trade_id;`	`SELECT trade_id, price, size FROM SILVER.TRADES QUALIFY ROW_NUMBER() OVER (PARTITION BY trade_id ORDER BY _updated_at DESC) = 1;`
P3	NOT IN with a NULL returns nothing	`SELECT symbol FROM GOLD.DIM_SYMBOLS WHERE symbol NOT IN (SELECT symbol FROM SILVER.BARS);`	`SELECT d.symbol FROM GOLD.DIM_SYMBOLS d WHERE NOT EXISTS (SELECT 1 FROM SILVER.BARS b WHERE b.symbol = d.symbol);`
P4	NULLs in aggregates and filters	`SELECT COUNT(sector) AS symbols FROM GOLD.DIM_SYMBOLS WHERE sector <> 'Technology';`	`SELECT COUNT(*) AS symbols FROM GOLD.DIM_SYMBOLS WHERE sector IS DISTINCT FROM 'Technology';`
P5	FLOAT used for money	`SELECT symbol, SUM(price::FLOAT * size) AS notional FROM SILVER.TRADES GROUP BY symbol;`	`SELECT symbol, SUM(price * size)::NUMBER(18, 4) AS notional FROM SILVER.TRADES GROUP BY symbol;`
P6	LIMIT or MAX_BY without a deterministic order	`SELECT symbol, volume FROM GOLD.FCT_BARS ORDER BY volume DESC LIMIT 10;`	`SELECT symbol, bar_ts, volume FROM GOLD.FCT_BARS ORDER BY volume DESC, symbol, bar_ts LIMIT 10;`
P7	DATEDIFF counts boundaries, not elapsed time	`SELECT DATEDIFF('hour', trade_ts, _loaded_at) AS hours_late FROM SILVER.TRADES;`	`SELECT DATEDIFF('second', trade_ts, _loaded_at) / 3600.0 AS hours_late FROM SILVER.TRADES;`
P8	Week boundaries depend on session settings	`SELECT DATE_TRUNC('week', bar_ts) AS wk, SUM(volume) FROM GOLD.FCT_BARS GROUP BY wk;`	`-- ISO week (Monday start); DAYOFWEEKISO ignores WEEK_START SELECT DATEADD('day', 1 - DAYOFWEEKISO(bar_ts), DATE_TRUNC('day', bar_ts)) AS wk, SUM(volume) FROM GOLD.FCT_BARS GROUP BY wk;`
P9	Quoted identifiers are case-sensitive	`-- illustrative table CREATE TABLE SILVER.EXTRA ("Symbol" VARCHAR, "Price" NUMBER); SELECT symbol FROM SILVER.EXTRA;  -- invalid identifier`	`-- illustrative table CREATE TABLE SILVER.EXTRA (symbol VARCHAR, price NUMBER); SELECT symbol FROM SILVER.EXTRA;`
P10	Semi-structured (VARIANT) traps	`-- illustrative table with a VARIANT column v SELECT v:symbol, v:price FROM BRONZE.RAW_JSON, LATERAL FLATTEN(input => v:trades) WHERE v:price > 100;`	`-- illustrative table with a VARIANT column v SELECT v:symbol::VARCHAR AS symbol, f.value:price::NUMBER(12,4) AS price FROM BRONZE.RAW_JSON, LATERAL FLATTEN(input => v:trades, OUTER => TRUE) f WHERE f.value:price::NUMBER(12,4) > 100;`
P11	Division by zero	`SELECT symbol, (close - open) / open AS bar_return FROM SILVER.BARS;`	`SELECT symbol, DIV0NULL(close - open, open) AS bar_return FROM SILVER.BARS;`
P12	UPDATE/MERGE with duplicate source rows	`UPDATE SILVER.SYMBOLS s SET sector = r.sector FROM BRONZE.RAW_SYMBOLS r WHERE r.symbol = s.symbol;`	`UPDATE SILVER.SYMBOLS s SET sector = r.sector FROM (SELECT symbol, sector FROM BRONZE.RAW_SYMBOLS QUALIFY ROW_NUMBER() OVER (PARTITION BY symbol ORDER BY _loaded_at DESC) = 1) r WHERE r.symbol = s.symbol;`
P13	String comparison surprises	`SELECT * FROM BRONZE.RAW_BARS WHERE symbol = 'aapl' AND volume > '1000';`	`SELECT * FROM SILVER.BARS WHERE symbol = UPPER(TRIM('aapl')) AND volume > 1000;`
P14	Window frames and partitions	`SELECT symbol, bar_ts, close / LAG(close) OVER (ORDER BY bar_ts) - 1 AS ret FROM GOLD.FCT_BARS;`	`SELECT symbol, bar_ts, close / LAG(close) OVER (PARTITION BY symbol ORDER BY bar_ts) - 1 AS ret FROM GOLD.FCT_BARS;`

#	Problem	Mistake	Fix
Q2	No partition pruning	`SELECT * FROM GOLD.FCT_BARS WHERE TO_VARCHAR(bar_ts, 'YYYY-MM-DD') = '2026-10-01';`	`SELECT * FROM GOLD.FCT_BARS WHERE bar_ts >= '2026-10-01' AND bar_ts < '2026-10-02';`
Q4	Exploding or cartesian joins	`SELECT b.symbol, s.sector FROM GOLD.FCT_BARS b, GOLD.DIM_SYMBOLS s;`	`SELECT b.symbol, s.sector FROM GOLD.FCT_BARS b JOIN GOLD.DIM_SYMBOLS s ON s.symbol = b.symbol;`
Q9	Reading columns you don't need	`SELECT * FROM GOLD.FCT_TRADES WHERE symbol = 'AAPL';`	`SELECT trade_ts, price, size FROM GOLD.FCT_TRADES WHERE symbol = 'AAPL';`
Q10	Thousands of tiny queries	`-- run once per symbol from Python SELECT MAX(bar_ts) FROM GOLD.FCT_BARS WHERE symbol = %s;`	`SELECT symbol, MAX(bar_ts) FROM GOLD.FCT_BARS GROUP BY symbol;`

How each agent is trained and tested

Scenarios reuse what the repo already has: generate_sample_data.py and generate_sample_news.py produce clean data, a small fault-injection layer corrupts it in known ways, and replay_sample_data.py plays it through Kafka. Every scenario has an expected diagnosis, so it can be scored.

Agent	Knowledge it is given	Training scenarios	Pass criteria (evals)
WATCH	Runbooks for each stage of ingestion; market calendar rules; Kafka/Connect error catalogue.	Fault injection with replay_sample_data.py: kill producer, drop minutes, duplicate bursts, malformed rows, pause connector.	Detects fault within N minutes; correct root cause; correct runbook step; zero alerts on closed-market periods.
DOCTOR	Task/stream/dbt failure catalogue with fixes; dbt DAG.	Break things on a dev account: let a stream go stale, suspend a task, insert duplicate keys, fail a dbt test.	Correct classification; failing rows returned; never weakens a test.
DQ	Expected shapes: 390 bars per session, price bands, null-rate baselines, corporate-action rules.	Inject splits, bad prints, gaps, halts, timezone shifts, clock skew, ms truncation, late and out-of-order rows into generated sample data.	Precision/recall on injected faults; explained vs unexplained moves.
XFORM	AGENTS.md conventions; dbt style; schema contracts.	Historical schema changes replayed as tasks; known bugs (e.g. GOLD_GOLD, lost policy) as test cases.	PR passes CI; reviewer accepts without major changes.
ANALYST	Semantic model, metric definitions, delay/entitlement rules, citation rules.	Question bank: verified queries, ambiguous questions, adversarial questions, news questions. Plus self-service questions: 'where is X', permission errors, huge export requests, mismatches with another system.	Numeric answers match sql/08 twins; routing accuracy; citation present; correct clarification rate.
CURATOR	Both semantic definitions; audit log; feedback table.	Seed failed/unanswered questions; seed YAML vs semantic-view mismatches.	Mismatch detection rate; accepted metric PRs.
GUARD	Policy model (who sees real-time); cost budgets; injection patterns.	Add a grant, create an unprotected table, plant injection text in sample_news.csv, run a cost spike, query the policy under different session timezones. Plus: an unused grant, an object owned by a departed user's role, ACCOUNTADMIN daily use, an idle warehouse, untagged queries.	Every planted violation detected; no false sev-1.
SCALE	Time-of-day volume baselines; macro-event calendar; capacity limits of each component.	replay_sample_data.py at 10x-50x --speedup; stop the Kafka container; suspend a warehouse; fill a test broker's disk.	Lag recovers within the agreed time; no credit overrun; no data lost; catch-up paced.
SQLCOACH	Sections P and Q of this catalog with their bad/good SQL pairs; Snowflake SQL semantics (NULLs, VARIANT, windows, DATEDIFF, pruning).	Each bad_sql runs on a clone seeded so the bug changes the answer (a NULL in a subquery, duplicate news rows, ties at the LIMIT boundary); slow queries built to spill or skip pruning.	Names the right problem ID; rewrite returns exactly the good_sql result; partitions scanned or bytes spilled go down for Q problems.
RELEASE	Environment layout, migration conventions, dependency order, rollback plans, AGENTS.md rules.	This repo's real history replayed as pull requests (GOLD_GOLD schema, wrong run order, policy lost on rebuild, session-timezone bug); drift injected in DEV; a column rename with live consumers.	Catches each bug before merge; deploy order proven on an empty database; impact analysis lists every affected consumer.
TRIAGE	Dependency graph of the stack; escalation policy.	Composite incidents built from the scenarios above.	First broken link identified; one incident per root cause.

Feedback loop in production

- Every agent action and answer is logged (AGENT_QUERY_AUDIT_LOG for ANALYST; an incident log for the others).
- Humans mark answers or actions as right or wrong; wrong ones become new eval cases with the corrected answer.
- The eval suite runs on every change to an agent's instructions, tools or the semantic model. A regression blocks the change.
- Agents move up an autonomy level only after a sustained pass rate on their evals and in production review.

Building the agents on Snowflake

The agents don't need a separate platform. Each piece maps to a Snowflake feature, so detection, action, approval, logging and evaluation all sit next to the data they protect.

Piece	Snowflake feature	Why
Triggers	Snowflake ALERT objects (CREATE ALERT ... IF (EXISTS (query)) THEN ...) on a schedule; Data Metric Functions attached to Bronze, Silver and Gold tables	Detects the condition inside Snowflake and wakes the right agent. No external scheduler is needed for in-Snowflake problems.
Agent brain	Cortex Agents (the same mechanism as MARKET_DATA_AGENT), or an external agent calling Snowflake through the repo's MCP server	ANALYST and CURATOR fit Cortex Agents naturally. WATCH has to reach Kafka and Alpaca, so it runs outside and writes its findings into Snowflake.
Tools	Stored procedures, one per runbook step (for example RESUME_TASK_SAFE, RECREATE_STREAM_WITH_BACKFILL), owned by a dedicated OPS role	The agent can only do what a procedure allows; the procedure checks preconditions and logs every call.
Approval	An OPS.PENDING_ACTIONS table plus a Streamlit in Snowflake page where a human approves or rejects	Implements L2 autonomy: the agent proposes, a human approves, the procedure runs.
Observability	Event table (Snowflake Trail) for procedure and agent logs; ACCOUNT_USAGE and INFORMATION_SCHEMA views	One place to reconstruct every incident and every agent decision.
Notification	Notification integration (email, or a webhook to Slack)	Alerts and approval requests reach humans without another system.
Evals	OPS.EVAL_CASES and OPS.EVAL_RESULTS tables; scenarios replayed into a zero-copy CLONE of the database	Fault scenarios run against a clone, so training runs never touch production.
Security	A separate least-privilege role per agent (OPS_WATCH, OPS_DOCTOR, ...); ANALYST_AGENT stays read-only	A wrong or compromised agent can only do what its role allows.

One problem end to end in Snowflake: a stale stream (D1)

- **Detect.** A scheduled ALERT checks SHOW STREAMS output for any stream whose stale_after date is less than 2 days away.
- **Diagnose.** The alert calls DOCTOR, which reads TASK_HISTORY and finds the consuming task has been suspended since a failure 9 days ago.
- **Propose.** DOCTOR writes a row to OPS.PENDING_ACTIONS: fix the task error, resume it, and consume the stream before it goes stale. Evidence attached.
- **Approve.** The on-call engineer approves it on the Streamlit in Snowflake page.
- **Act.** The RESUME_TASK_SAFE procedure checks preconditions, resumes the task, and logs to the event table.
- **Verify.** DQ confirms Silver row counts have caught up with Bronze. The incident closes, and the case is added to OPS.EVAL_CASES so the next DOCTOR version is tested on it.

Rollout order

Built in the order that matches the repo's README plan, so each step has real infrastructure to test against:

- **Step 0: prerequisites.** Connect the live Snowflake trial, local Kafka, GCS bucket and Alpaca key; run every SQL file for real; build the pytest suite and CI (README “Target” items). Agents can't be evaluated against a pipeline that has never run.
- **Step 1: fault-injection harness.** Extend the sample generators so each problem in this document can be triggered on demand, and seed test data so every bad/good SQL pair gives a different answer.
- **Step 2: L0/L1 observers.** WATCH, DQ, GUARD, SCALE (observe only) and SQLCOACH (review only) first, read-only. They catch the problems that silently damage data.
- **Step 3: analyst side.** ANALYST evals against the sql/08 twins, then CURATOR for the semantic model.
- **Step 4: repair agents.** DOCTOR, XFORM, RELEASE and SCALE actions (scaling, catch-up, deploys), approval-gated (L2).
- **Step 5: TRIAGE** once the specialists are reliable enough to call as tools.